



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Escola d'Enginyeria de Barcelona Est

Project Report

Movement Simulation

Dynamic optimization of a Stewart Platform



By Joan Comabella Llavina
Tutor: Gil Serrancolí Masferrer

List of contents

Introduction.....	3
Work developed.....	4
Results.....	7
Conclusion.....	11
Annex.....	12

Introduction

This project serves as a capstone exercise to apply the theoretical and practical knowledge acquired during the course to optimize the platform's dynamic behavior during a prescribed motion. The goal is to design an optimal control strategy that moves the platform from an initial configuration to a desired final state while adhering to physical and operational constraints.

Dynamic Optimization is formulated as an optimal control problem, where the objective is to minimize a predefined cost function. This cost function may represent criteria such as energy consumption, actuator effort, or trajectory tracking error. The optimization process uses advanced techniques such as Direct Collocation (DC) and leverages numerical computation libraries like CasADi integrated with MATLAB. These methods enable precise and efficient solutions to high-dimensional control problems by discretizing the motion equations and solving them as nonlinear programming tasks.

The Stewart Platform, known for its six degrees of freedom (DoF), provides a rich testbed for dynamic optimization due to its highly coupled and constrained kinematic and dynamic structure. The model includes three translational and three rotational degrees of freedom, achieved through six actuators that connect the base to the moving platform. The optimization framework accounts for constraints such as actuator limits, collision avoidance, and adherence to the dynamic equations of motion, ensuring the feasibility and effectiveness of the generated control strategies.

This project highlights the application of dynamic optimization to a real-world engineering challenge. By integrating robust numerical methods and leveraging the unique characteristics of the Stewart Platform, the project offers a comprehensive approach to analyzing and enhancing the performance of complex robotic systems.

Work developed

The model selected for this project is based on the Stewart Platform, a six-degrees-of-freedom (DOF) parallel robotic mechanism. This system consists of six actuators that control the motion of a moving platform relative to a fixed base, using a series of prismatic joints connected via spherical and universal joints. The platform's configuration is defined by three translational coordinates (tx,ty,tz) and three rotational coordinates(q1,q2,q3), enabling full spatial motion.

In this model, the optimization focuses on controlling the platform's motion from an initial to a desired final configuration, while minimizing a specific cost function, such as actuator effort or energy consumption. The optimization problem is formulated using dynamic equations of motion as constraints, ensuring that the generated trajectory is physically and dynamically feasible. Additional constraints, such as actuator limits and collision avoidance, are also incorporated to reflect real-world scenarios.

While the Stewart Platform offers six degrees of freedom for motion, it operates under significant constraints due to actuator limitations. This makes it a dynamically under-actuated system in certain configurations, where the optimization solver must balance competing objectives to achieve an optimal solution. As a result, the computation of the optimal trajectory involves a complex trade-off between precision, energy efficiency, and mechanical feasibility, making the Stewart Platform a challenging yet insightful model for dynamic optimization tasks.

In the simulation, a vector named FIDj is generated. This vector contains information about the moments, forces, and positional coordinates of various points in the system. Based on this output, a function was developed to calculate the forces exerted by each actuator individually. The process involves several steps, outlined as follows:

$$FIDj = [M'_{z'}, M'_{x'}, M'_{y'}, F_{x'}, F_{y'}, F_{z'}, Q_{1_x}, Q_{1_y}, Q_{1_z}, Q_{2_x}, Q_{2_y}, Q_{2_z}, Q_{3_x}, Q_{3_y}, Q_{3_z}]$$

1. Transforming Moments to Cartesian Coordinates
 - a. The moments provided in the vector FIDj are initially represented in an Euler angle basis. To make these moments compatible with further calculations, they are transformed into a Cartesian coordinate basis. This transformation ensures that all the data is expressed in the same reference frame, simplifying subsequent computations.

$$M_z = \begin{bmatrix} 0 \\ 0 \\ M'_z \end{bmatrix} \quad M_x = \begin{bmatrix} \cos(\psi) & -\sin(\psi) & 0 \\ \sin(\psi) & \cos(\psi) & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} M'_x \\ 0 \\ 0 \end{bmatrix} \quad M_y = \begin{bmatrix} \cos(\psi) & -\sin(\psi) & 0 \\ \sin(\psi) & \cos(\psi) & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos(\theta) & -\sin(\theta) \\ 0 & \sin(\theta) & \cos(\theta) \end{bmatrix} \cdot \begin{bmatrix} 0 \\ M'_y \\ 0 \end{bmatrix}$$

2. Calculating Unit Vectors for Each Actuator

- a. To determine the forces applied by the actuators, we need to establish the direction of each actuator relative to the system. This is achieved by calculating the unit vectors of the actuators:
 - i. The unit vectors are computed with respect to the fixed points on the base and the connection points on the moving platform.
 - ii. For this simulation, we selected a configuration where two actuators share the same connection point on the base. The six actuators are arranged in such a way that they form three triangular configurations, ensuring structural stability and efficient force distribution.

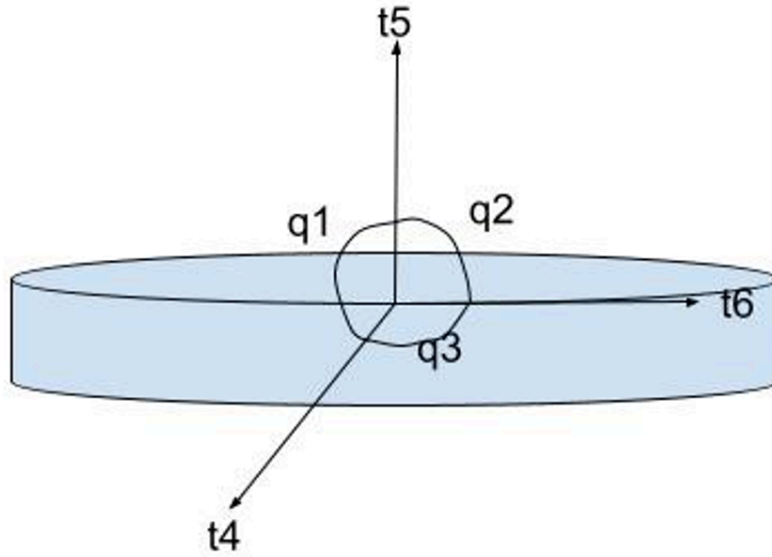
3. Setting Up a System of Equations

- a. Once the unit vectors for all actuators are determined, a system of six equations with six unknowns is constructed.

$$\sum_{i=1}^6 \bar{\lambda}_i \cdot F_i = \bar{F}$$

$$\sum_{i=1}^6 \bar{M}_i = \bar{M}$$

To understand how we establish the lower and upper bounds, it is essential to first explore how movements are defined in OpenSim. In this context, qq represents the rotational degrees of freedom, while tt corresponds to the translational degrees of freedom. These bounds are determined based on the range of motion allowed for each joint or segment in the simulation. By analyzing the specific constraints and capabilities of the system, we can accurately define the limits for both rotations and translations, ensuring realistic and precise simulations.



States bounds during the movement		
Coordinate	States lower bound	States upper bound
q1	-90°	90°
q2	-90°	90°
q3	-90°	90°
t4	-2	2
t5	0	2
t6	-2	2

$$w_{limit} = \frac{\pi}{18} \left(\frac{rad}{s} \right)$$

$$v_{limit} = 10 \text{ (m/s)}$$

Finally, the solver needs to have an objective to minimize, referred to as the cost function. Initially, this cost function is defined as the sum of joint moments.

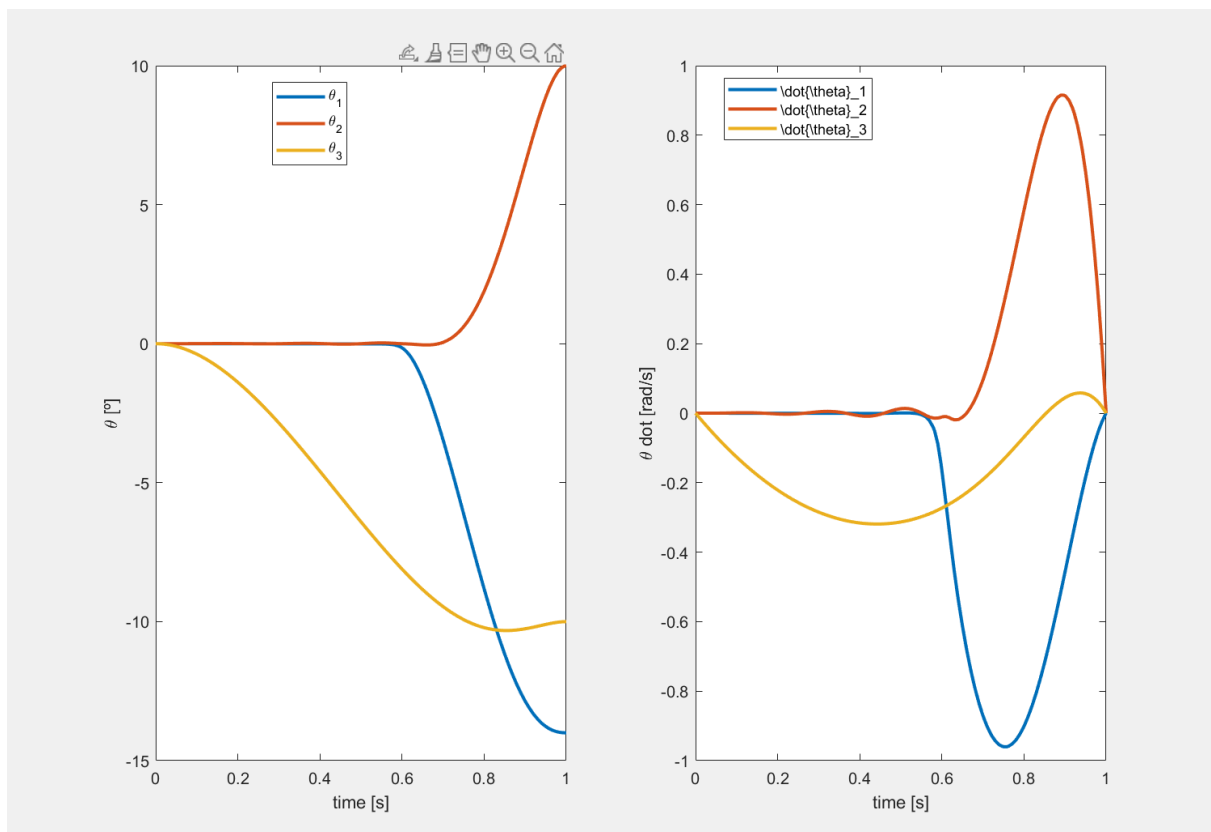
However, the resulting movement does not accurately reflect realistic human walking kinematics. Therefore, alternative cost functions can be introduced to achieve more realistic motion, such as the sum of coordinate accelerations or the sum of moments exerted by the actuators.

$$L_1 = \sum_{n=1}^6 M_n^2 \quad L_2 = \sum_{n=1}^6 \alpha_n^2$$

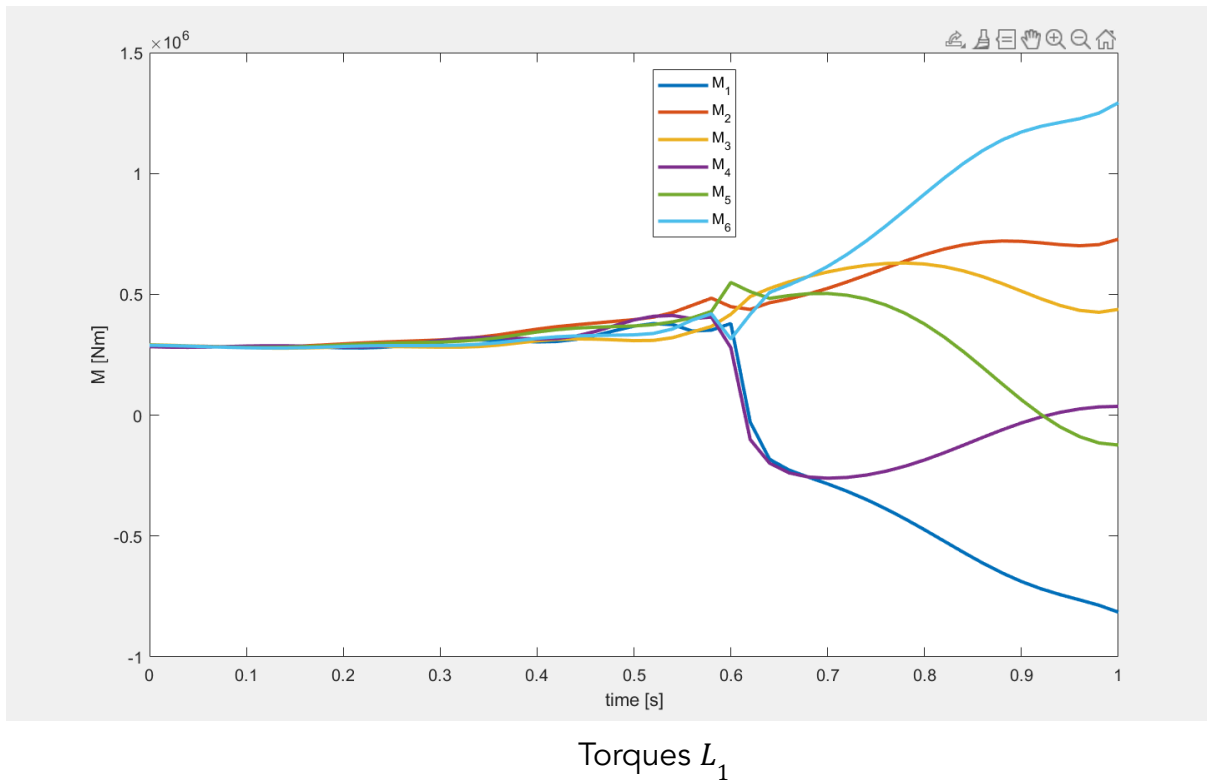
Another cost function we considered was the total power generated, calculated as the sum of joint moments multiplied by their respective angular velocities. However, this approach also failed to find an optimal solution. Even when we further increased the lower and upper bounds for the variables, the solver was unable to converge to a feasible or realistic result. This suggests that relying solely on power as a cost function may not adequately guide the system toward a realistic or optimal motion pattern.

Results

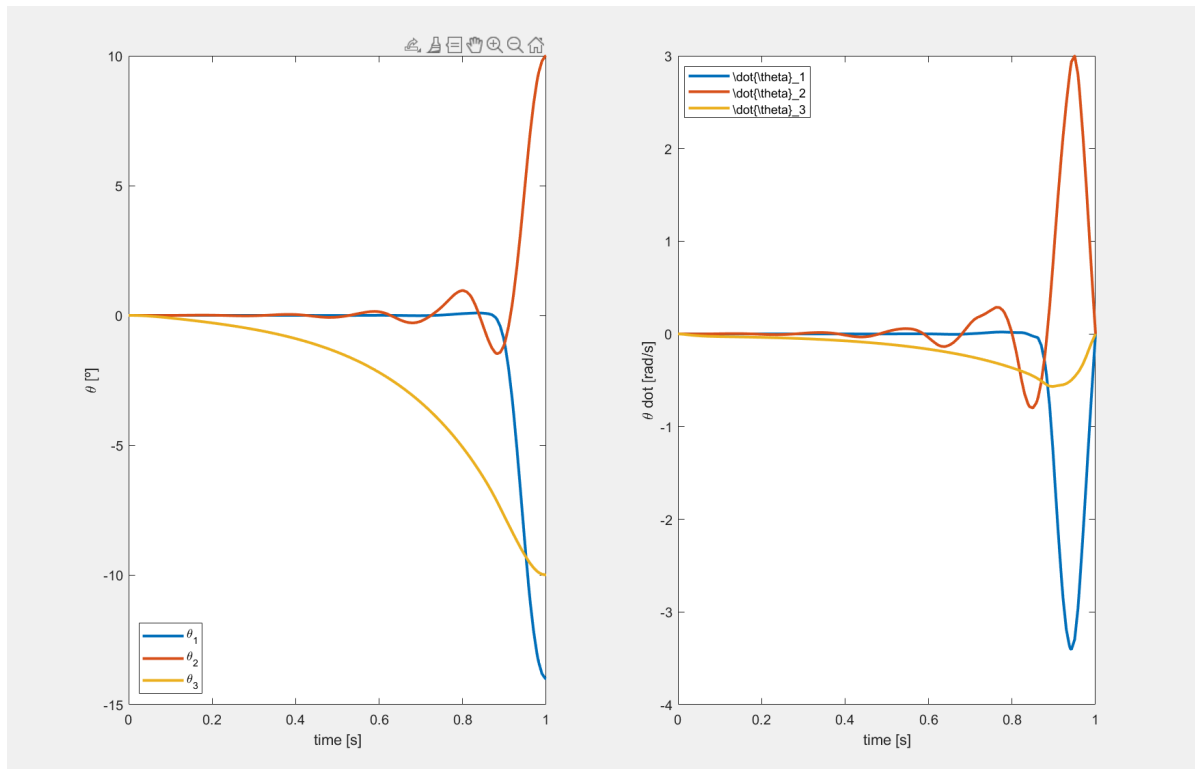
The results obtained on the simulations are quite unique. For the 1st, the cost function was just minimizing the joint moments and solution obtained is:



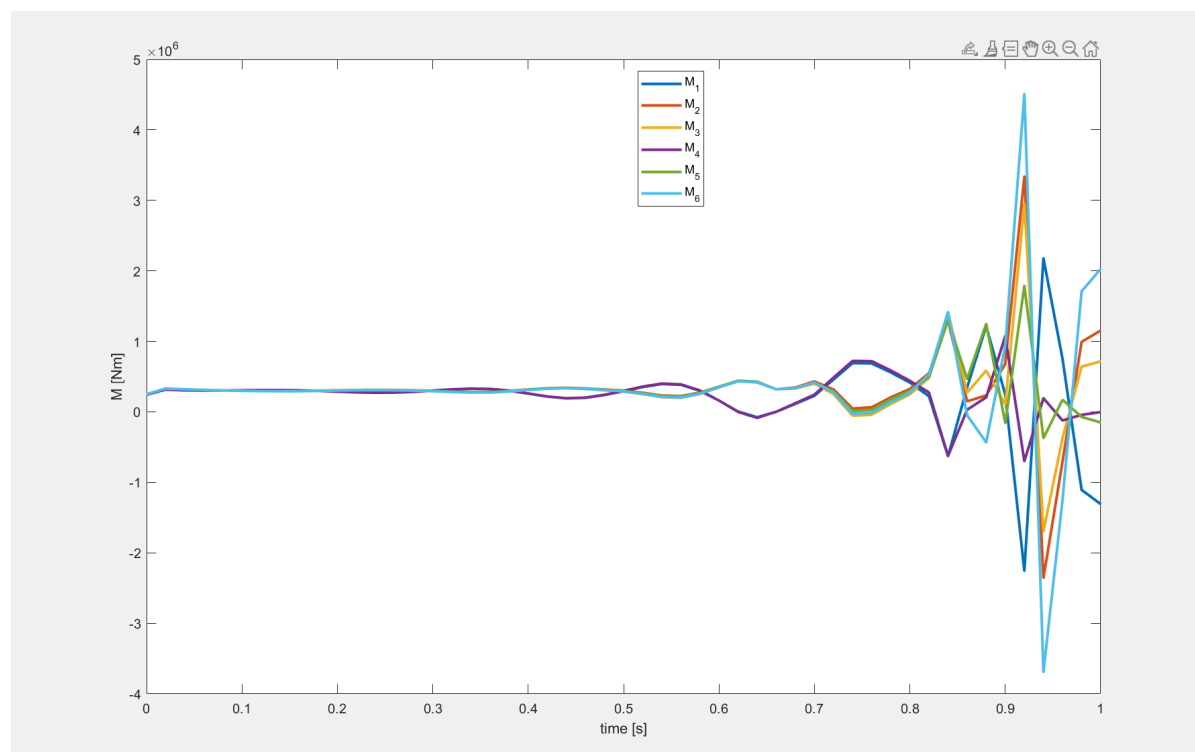
Coordinates and velocities L_1



The results have been highly satisfactory, as the approach leads directly to the final solution without any disturbances or actions that alter the orientation of the platform. This ensures that the system achieves its goal efficiently and with precision, maintaining stability and avoiding unnecessary deviations throughout the process.



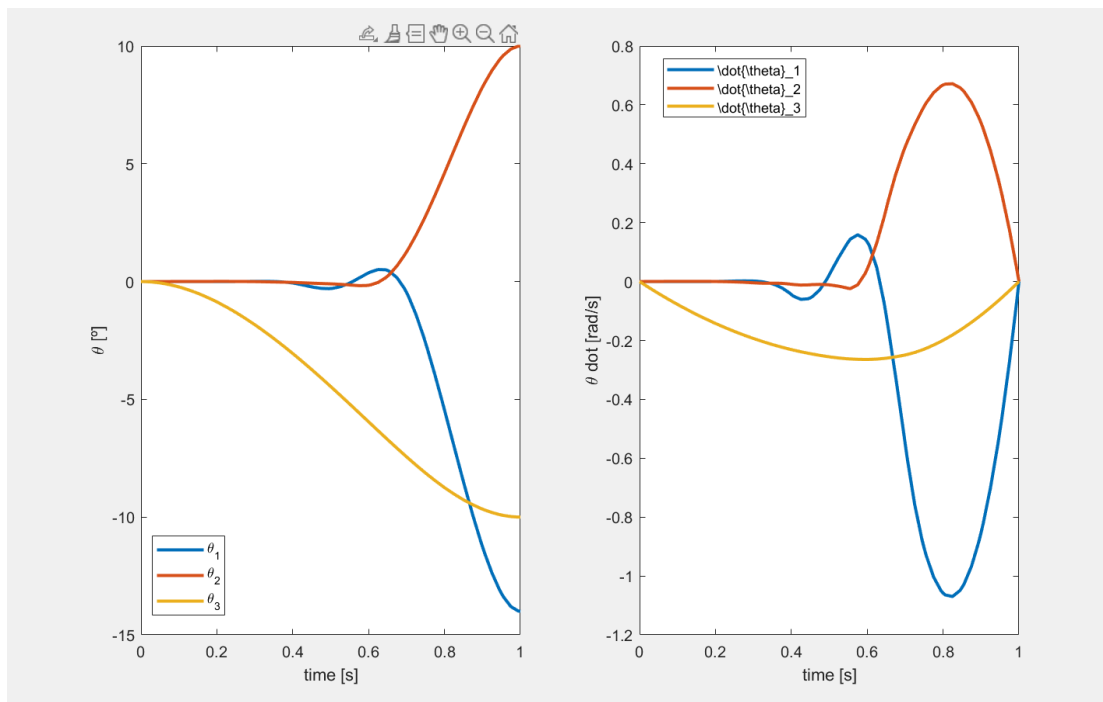
Coordinates and velocities L_2



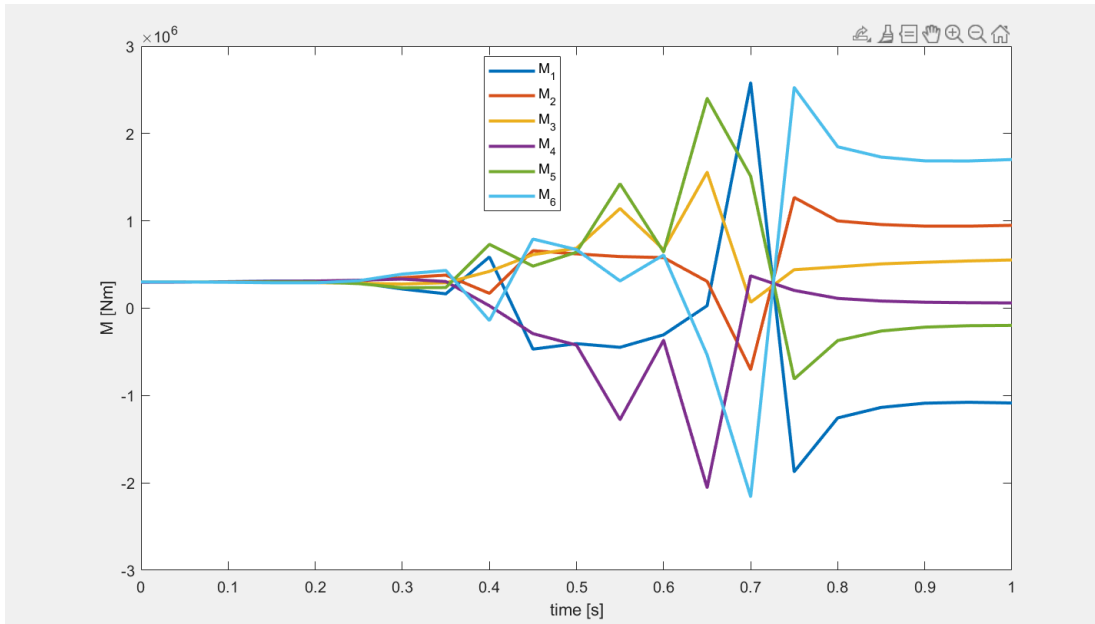
Torques L_2

For this optimization, it was necessary to increase the lower and upper bounds of the variables to allow the solver more flexibility in finding a feasible solution. This adjustment provided a wider range for the system to explore during the optimization process. Despite this, the results demonstrated that even with expanded bounds, certain cost functions led to undesired intermediate positions or unrealistic moment profiles, highlighting the challenges of balancing constraints with achieving realistic and practical motion patterns for the Stewart platform.

These second results are not as satisfactory, as fulfilling the cost function requires the platform to pass through undesired positions. This affects the ultimate goal of accurately simulating realistic positions of airplanes or helicopters. Such deviations undermine the realism and practicality of the simulation, highlighting the limitations of this approach in achieving the intended objective.



Coordinates and velocities $L_1 + L_2$



$$\text{Torques } L_1 + L_2$$

These latest results represent an improvement over the second cost function; however, I would still not recommend using this approach. The final outcomes for the joint moments exhibit an unusual linearity that seems unrealistic for the intended application. This raises concerns about the validity of the solution, particularly when the goal is to replicate realistic behaviors for simulating airplane or helicopter positions.

Conclusion

In conclusion, the Stewart platform movement simulation project has been a significant success. The primary objective was to become proficient in MATLAB and the CasADi framework while effectively applying these tools to the dynamic optimization problem of the platform. This project has provided valuable experience in modeling and optimization techniques, particularly in formulating kinematics and solving dynamic optimization problems using direct collocation (DC).

The exploration of cost functions (L1, L2, L3) highlighted key insights into the platform's behavior. Among these, the first optimization (L1) proved to be the most effective, offering stable and realistic solutions without introducing undesired perturbations or deviations. This aligns well with the primary goal of simulating realistic positions and orientations, crucial for mimicking the movement of aircraft and helicopters.

While alternative cost functions showed potential improvements, such as reducing power or minimizing accelerations, their limitations such as undesired intermediate positions or overly linear moment profiles highlight the robustness and reliability of L1. This emphasizes the importance of choosing cost functions aligned with the project's end goals.

Overall, this project underscores the power of digital simulations in achieving precise and realistic motion control for Stewart platforms. It demonstrates not only the efficiency of dynamic optimization methods but also their potential for advancing cost-effective and accurate solutions in aerospace and robotics applications.

Annex

Constant parameters values for the optimization simulations

```
% Time horizon
T = 1.0;

%Number of coordinates and torque controls
nq=6;
ncT=6;

%Geometric and dynamics parameters
fixedPoints = [
    360, 0, 0;    % 01
    180, 0, 311.77; % 02
    -180, 0, 311.77; % 03
    -360, 0, 0;    % 04
    -180, 0, -311.77; % 05
    180, 0, -311.77; % 06
];

auxdata.g=9.81;

% Declare model variables
q1 = MX.sym('q1');
q2 = MX.sym('q2');
q3 = MX.sym('q3');
q4 = MX.sym('q4');
q5 = MX.sym('q5');
q6 = MX.sym('q6');
q1dot = MX.sym('q1dot');
q2dot = MX.sym('q2dot');
q3dot = MX.sym('q3dot');
q4dot = MX.sym('q4dot');
q5dot = MX.sym('q5dot');
q6dot = MX.sym('q6dot');
x=[q1 q1dot q2 q2dot q3 q3dot q4 q4dot q5 q5dot q6 q6dot];
uT1 = MX.sym('uT1');
uT2 = MX.sym('uT2');
uT3 = MX.sym('uT3');
uT4 = MX.sym('uT4');
uT5 = MX.sym('uT5');
uT6 = MX.sym('uT6');
uT=[uT1 uT2 uT3 uT4 uT5 uT6];
ua_q1=MX.sym('ua_q1');
ua_q2=MX.sym('ua_q2');
ua_q3=MX.sym('ua_q3');
ua_q4=MX.sym('ua_q4');
ua_q5=MX.sym('ua_q5');
ua_q6=MX.sym('ua_q6');
ua=[ua_q1 ua_q2 ua_q3 ua_q4 ua_q5 ua_q6];

% Model equations
F=external('F','stewardplatform.dll');
xdot = [q1dot;ua_q1; q2dot; ua_q2; q3dot; ua_q3;q4dot;ua_q4; q5dot; ua_q5; q6dot; ua_q6];

s.uT=1e6;
s.ua=100;

% Cost function
%L = sum((uT).^2)+0.01*sum((ua).^2);
% L = sum(uT.*(x(8:2:nq*2)))+0.01*sum((ua).^2);
% L = sum(ua.^2)+0.01*sum((uT).^2);
L = sum((uT).^2)+sum(ua.^2);
```

```

% Boundary conditions
%fixed initial states
initial_states_lb=[      0;   0;           0;   0;   0;   0; 0; 0; 0; 1;   0; 0; 0; 0]; %q1 q1dot q2 q2dot q3
initial_states_ub=[      0;   0;           0;   0;   0;   0; 0; 0; 0; 1;   0; 0; 0; 0];
initial_states_ig=[      0;   0;           0;   0;   0;   0; 0; 0; 0; 1;   0; 0; 0; 0];
%fixed final states
final_states_lb=[-14*2*pi/360;   0; 10*2*pi/360;   0; -10*pi/180;   0; 0;   0;   1;   0; 0; 0; 0];
final_states_ub=[-14*2*pi/360;   0; 10*2*pi/360;   0; -10*pi/180;   0; 0;   0;   1;   0; 0; 0; 0];
final_states_ig=[-14*2*pi/360;   0; 10*2*pi/360;   0; -10*pi/180;   0; 0;   0;   1;   0; 0; 0; 0];
%state bounds during the movement
states_lb=[      -pi/2; -10;           -pi/2; -10;   -pi/2; -10; -2; -10; 0; -10; -2; -10];
states_ub=[       pi/2; 10;           pi/2; 10;    pi/2; 10; 2; 10; 2; 10; 2; 10];
states_ig=[        0;   0;           0;   0;   0;   0; 0; 0; 0; 1;   0; 0; 0; 0];
%control bounds
controls_lb=-1e8*ones(nq,1)/s.uT;
controls_ub= 1e8*ones(nq,1)/s.uT;
controls_ig=zeros(nq,1);
%acceleration bounds (implicit form)
accelerations_lb=-100*ones(nq,1)/s.ua;
accelerations_ub= 100*ones(nq,1)/s.ua;
accelerations_ig= zeros(nq,1);

% Control discretization
N = 20; % number of control intervals (20 intervals/second reasonable)

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

```

Function that calculate the force Values

```

function [unitVectors, forceValues, cg] = calculateUnitVectors(fIDj, fixedPoints,eul)
import casadi.*

if length(fIDj) ~= 15
    error('fIDj debe tener 15 componentes.');
```

```

end
if size(fixedPoints, 1) ~= 6 || size(fixedPoints, 2) ~= 3
    error('fixedPoints debe ser una matriz 6x3 con las coordenadas de los puntos fijos.');
```

```

end

Rz = fIDj(1); |
Rx = fIDj(2);
Ry = fIDj(3);
Fx = fIDj(4);
Fy = fIDj(5);
Fz = fIDj(6);
Q1 = fIDj(7:9); % [Q1x, Q1y, Q1z]
Q2 = fIDj(10:12); % [Q2x, Q2y, Q2z]
Q3 = fIDj(13:15); % [Q3x, Q3y, Q3z]
cg=(Q1+Q2+Q3)/3

% Extraer ángulos de Euler: psi, theta, phi
psi = eul(1); % Rotación alrededor del eje Z
theta = eul(2); % Rotación alrededor del eje X
phi = eul(3); % Rotación alrededor del eje Y

% Definir las matrices de rotación
Rz1 = [cos(psi), -sin(psi), 0;
       sin(psi), cos(psi), 0;
       0, 0, 1];

Rx1 = [1, 0, 0;
       0, cos(theta), -sin(theta);
       0, sin(theta), cos(theta)];

Ry1 = [cos(phi), 0, sin(phi);
       0, 1, 0;
       -sin(phi), 0, cos(phi)];

```

```

Mx=Rz1*[Rx;0;0];
My=Rz1*Rx1*[0;Ry;0];
Mz=[0;0;Rz];

M=Mx+My+Mz;

% Inicializar matriz de vectores unitarios
unitVectors = MX.zeros(6, 3);

% 01 y 02 hacia Q1
unitVectors(1, :) = (Q1 - fixedPoints(1, :))' / sqrt(sum((Q1 - fixedPoints(1, :)).^2)+1e-6);
unitVectors(2, :) = (Q1 - fixedPoints(2, :))' / sqrt(sum((Q1 - fixedPoints(2, :)).^2)+1e-6);

% 03 y 04 hacia Q2
unitVectors(3, :) = (Q2 - fixedPoints(3, :))' / sqrt(sum((Q2 - fixedPoints(3, :)).^2)+1e-6);
unitVectors(4, :) = (Q2 - fixedPoints(4, :))' / sqrt(sum((Q2 - fixedPoints(4, :)).^2)+1e-6);

% 05 y 06 hacia Q3
unitVectors(5, :) = (Q3 - fixedPoints(5, :))' / sqrt(sum((Q3 - fixedPoints(5, :)).^2)+1e-6);
unitVectors(6, :) = (Q3 - fixedPoints(6, :))' / sqrt(sum((Q3 - fixedPoints(6, :)).^2)+1e-6);

r1 = Q1 - cg; % Vector de cg a Q1
r2 = Q2 - cg; % Vector de cg a Q2
r3 = Q3 - cg; % Vector de cg a Q3

% 1. Ecuaciones de fuerzas
A_forces=unitVectors';

% 2. Ecuaciones de momentos (producto cruzado con r)
A_moments = [
    cross(r1, unitVectors(1, :)) , cross(r1, unitVectors(2, :)) , ...
    cross(r2, unitVectors(3, :)) , cross(r2, unitVectors(4, :)) , ...
    cross(r3, unitVectors(5, :)) , cross(r3, unitVectors(6, :))];

% Matriz de ecuaciones combinadas
A = [A_forces; A_moments];

% Vector de términos independientes
b = [Fx; Fy; Fz; M];

- - -

% Vector de términos independientes
b = [Fx; Fy; Fz; M];

% Resolver el sistema
forceValues = inv(A)* b;

```